



E – Ticket Bus: Online bus ticket reservation site

Group 7

Names : Kaila Whyli Titani
Nabil Muhammad Nur Khairy
Silvia Puspita Sari

Class : 2ISA1

CEP CCIT

FAKULTAS TEKNIK UNIVERSITAS INDONESIA

2025

PROJECT ON E – Ticket Bus: Online bus ticket reservation site

Developed by

- 1. Kaila Whyli Titani**
- 2. Nabil Muhammad Nur Khairy**
- 3. Silvia Puspita Sari**

E – Ticket Bus: Online bus ticket reservation site

Batch Code : 2ISA1

Start Date : 20 February 2025

End Date : 06 March 2025

Name of Faculty : Mr. Harry Prasetyo Rosianto, S.Tr.T

Names of Developer :

1. Kaila Whyli Titani
2. Nabil Muhammad Nur Khairy
3. Silvia Puspita Sari

Date of Submission: 06 March 2025

CERTIFICATE

This is to certify that this report titled “E – Ticket Bus: Online bus ticket reservation site” embodies the original work done by Muhammad Reva Mahesa Widodo, Muhammad Angkasa Khairul Anam and Abdurrahman Wafi Adzima Ramadhan. Project in partial fulfillment of their course requirement at CEP-CCIT FTUI.

Coordinator:

Mr. Harry Prasetyo Rosianto, S.Tr.T.

ACKNOWLEDGEMENT

First of all, We would like to thank God Almighty. Because of His grace and guidance, Author can complete and present this paper that contains about E – Ticket Bus: Online bus ticket reservation site Thanks to Mr. Harry Prasetyo Rosianto, S.Tr.T. , and other faculty who always help to finish this task. Also, thanks to other fellow students and fellow workers in CCIT FTUI who supported The Author.

We wish for this paper can be useful for readers so it can add knowledge and insight in the future. The Author realized this Project still imperfect in arrangement and the plan. Therefore, The Author really expects all suggestions and criticisms from readers who are constructive for perfection of this paper and for future task.

Finally, hopefully this paper can provide readers and The Author many benefits for both of readers and Authors.

BACKGROUND

The development of technology is increasingly rapid and provides many conveniences in various aspects of life, including in the transportation system. One of the challenges often faced by the public is the bus ticket booking process which is still done manually. This traditional method often causes various problems, such as long lines at the counter, limited information about departure schedules, and the risk of running out of tickets.

To overcome these problems, a digital-based bus ticket booking system is needed that can help prospective passengers make ticket reservations more easily and efficiently. This system will allow users to view departure schedules, select seats, and make payments online without having to come directly to the terminal. With this system, it is expected that the user experience in buying bus tickets will be more comfortable, fast, and practical.

In addition to providing convenience for passengers, this system also provides benefits for transportation companies in managing booking data in a more organized manner. The data stored in the system can be used to optimize bus operations, analyze passenger travel patterns, and improve service quality.

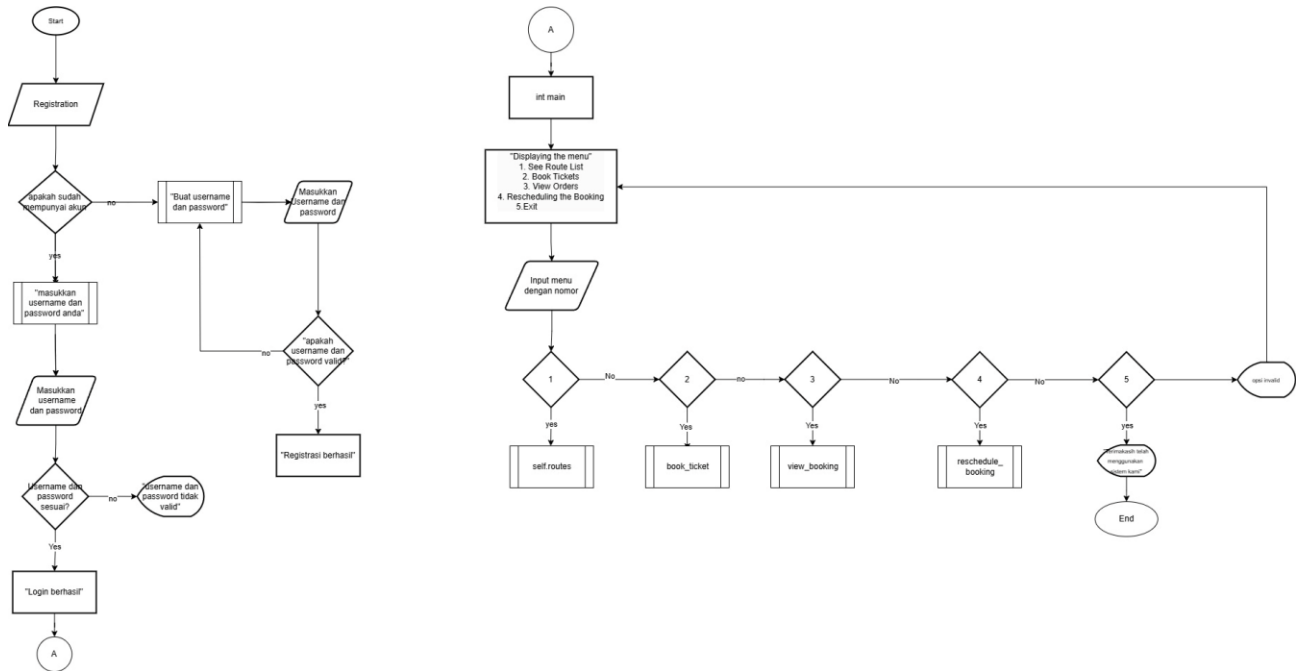
SYSTEM ANALYSIS

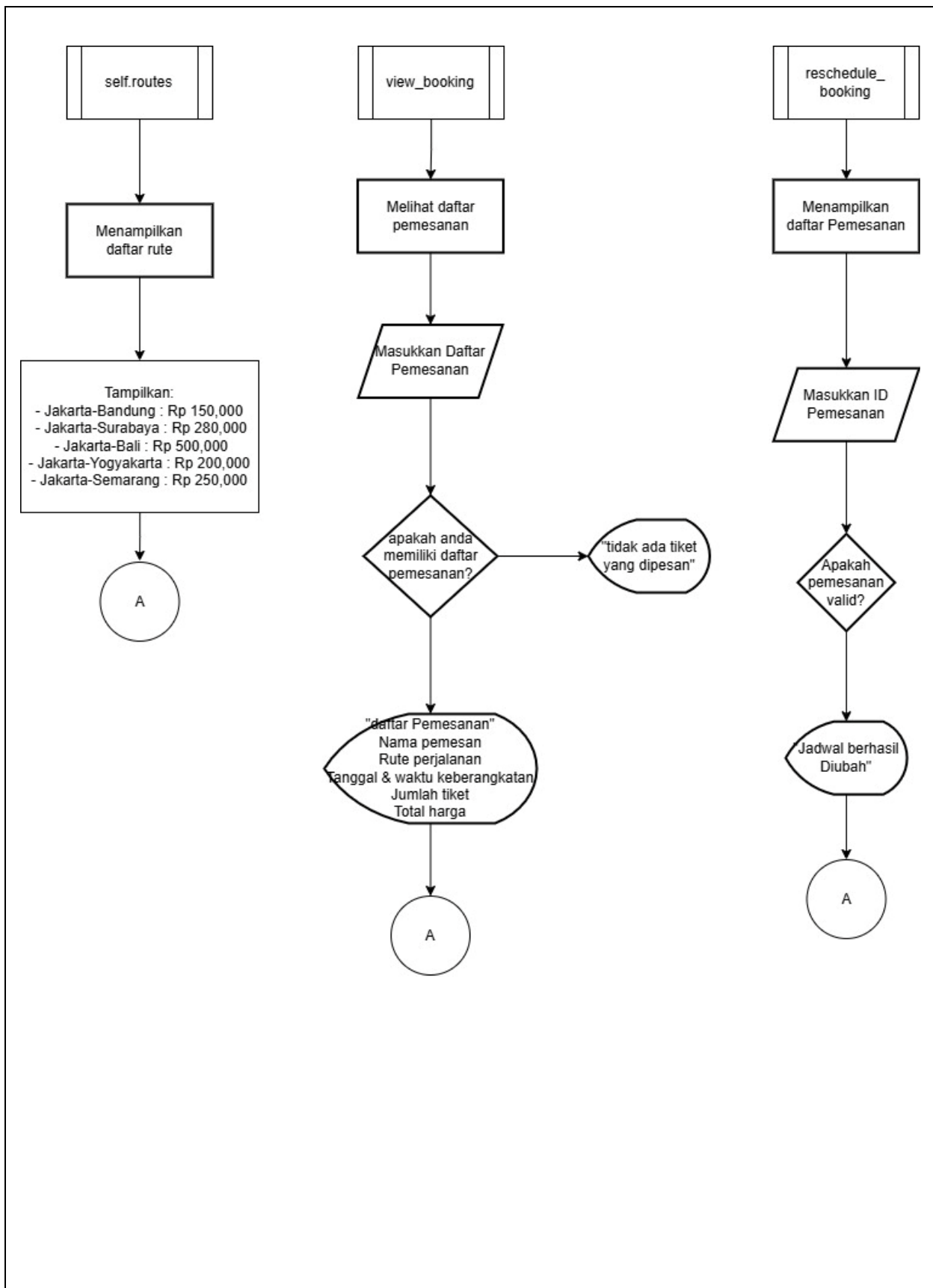
With a solid basis and fairly large functionality, this bus ticket reservation system offers viewing and editing bookings functionality, booking tickets with ample input validation, and registration and login with appropriate validation.

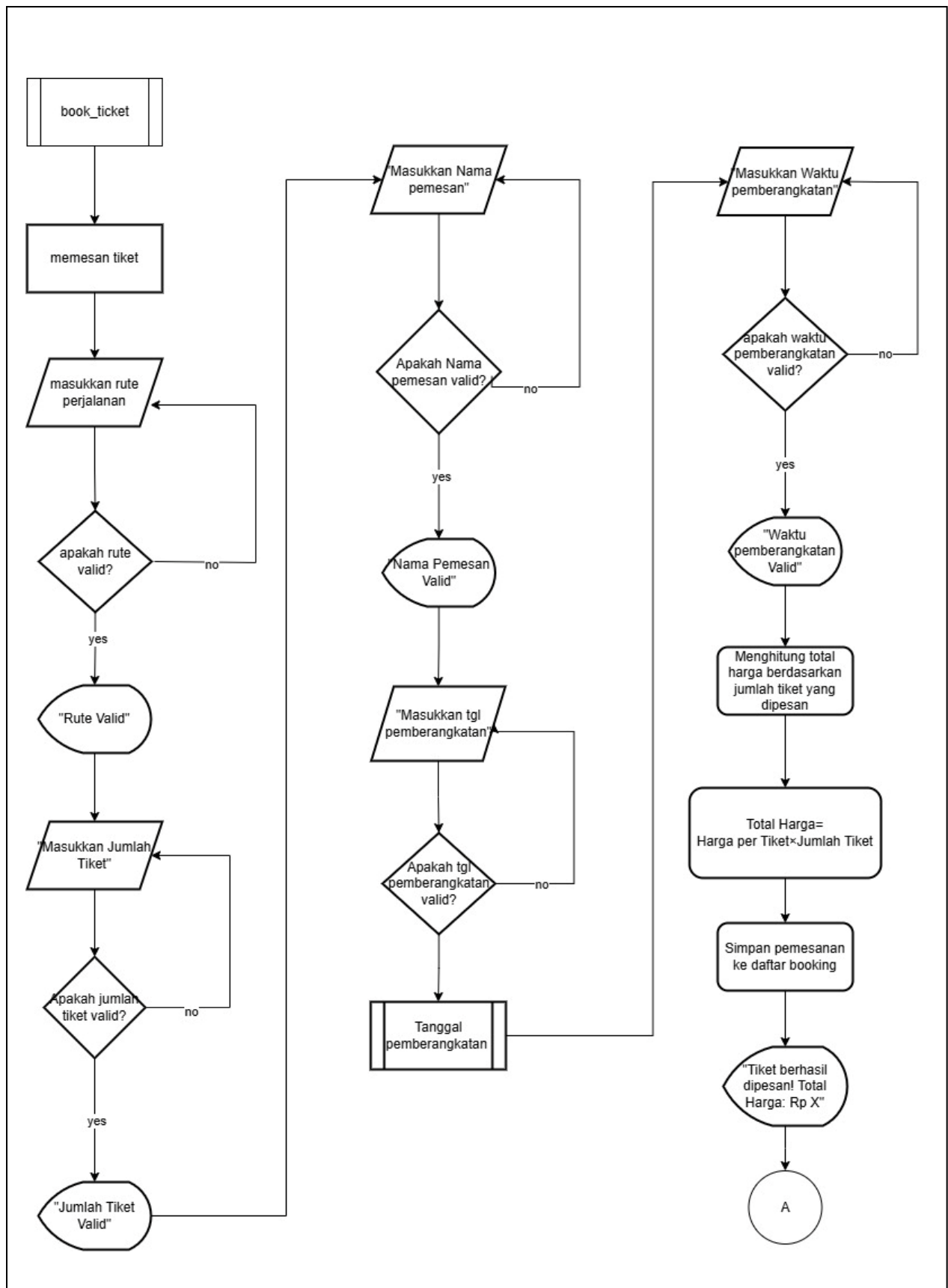
The good programming habits in date/time manipulation and input validation are seen in the use of the datetime module and regular expressions (regex). Booking details can be stored and managed efficiently with the help of data structures like dictionaries and lists.

There are certain aspects, however, that can be contrasted. Continuous data collection, electronic payment, registration for courses, and reminders are certain development which can add functionality and experience of the user. Also, system security can be increased using password hashing. In addition, Graphical User Interface (GUI) can aid in accommodating better user interaction. With additional improvement, this system can be a more efficient and user-friendly program for bus ticket booking application.

FLOWCHART







SOURCE CODE

```
import re
from datetime import datetime

class BusTicketBooking:
    def __init__(self):
        self.routes = {
            "Jakarta-Bandung": 150000,
            "Jakarta-Surabaya": 280000,
            "Jakarta-Bali": 500000,
            "Jakarta-Yogyakarta": 200000,
            "Jakarta-Semarang": 250000
        }
        self.bookings = []
        self.user = {
            "admin1": "Password2#",
            "admin2": "Password1@",
            "admin3": "Password3$",
            "admin4": "Password4%",
            "admin5": "Password("
        } #dictionary untuk menyimpan password

    def validate_username(self, username):
        pattern = r"^[a-zA-Z0-9._-]{5,15}$"
        return bool(re.match(pattern, username))

    def validate_password(self, password):
        pattern = r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%&*#_-])[A-Za-z\d@$!%&*#_-]{8,}$"
        return bool(re.match(pattern, password))

    def register(self):
        while True:
            username = input("Masukkan username baru: ")
            if self.validate_username(username):
                if username not in self.user:
                    break
            else:
```

```

        print("Username sudah digunakan. Silahkan coba lagi.")
    else:
        print("Username tidak valid. Silahkan coba lagi.")

    while True:
        password = input("Masukkan password baru: ")
        if self.validate_password(password):
            self.user[username] = password
            print("\nRegistrasi berhasil!")
            return
        else:
            print("Password tidak valid. Silahkan coba lagi.")

    def login(self):
        while True:
            username = input("Masukkan username: ")
            password = input("Masukkan password: ")
            if username in self.user and self.user[username] == password:
                print("Login berhasil!")
                return True
            else:
                print("Username atau password salah. Silahkan coba lagi.")
                return False

    def get_validate_input(self, prompt, validation_func, error_message):
        while True:
            value = input(prompt)
            if validation_func(value):
                return value
            else:
                print(error_message)

    def validate_ticket_count(self, ticket_count):
        return ticket_count.isdigit() and int(ticket_count) > 0

    def validate_name(self, name):
        return re.match(r"^[A-Za-z\s]+$", name)

    def book_ticket(self):
        #(fungsi book_ticket tetap sama)

```

```

pass #replace with your actual book_ticket implementation

def view_booking(self):
    #(fungsi view_booking tetap sama)
    pass #replace with your actual view_booking implementation

def reschedule_booking(self):
    #(fungsi reschedule_booking tetap sama)
    pass #replace with your actual reschedule_booking implementation

def display_routes(self):
    print("\nDaftar Rute dan Harga Tiket:")
    for index, route in enumerate(self.routes, start=1):
        print(f"{index}. {route}: Rp {self.routes[route]}")

def validate_date(self, date_text):
    try:
        date_obj = datetime.strptime(date_text, "%Y-%m-%d")
        if date_obj.date() < datetime.now().date():
            return False
        return True
    except ValueError:
        print("Format tanggal tidak valid. Gunakan formatendorong-MM-DD.")
        return False

def validate_time(self, time_text):
    try:
        datetime.strptime(time_text, "%H:%M")
        return True
    except ValueError:
        print("Format waktu tidak valid. Gunakan format HH:MM.")
        return False

def get_valid_input(self, prompt, validation_func, error_message):
    while True:
        value = input(prompt)
        if validation_func(value):
            return value
        else:
            print(error_message)

```

```

def validate_ticket_count(self, ticket_count):
    return ticket_count.isdigit() and int(ticket_count) > 0

def validate_name(self, name):
    return re.match(r"^[A-Za-z\s]+$", name)

def book_ticket(self):
    self.display_routes()
    selected_route = input("\nPilih rute (1/2/3/4/5): ")
    if selected_route in ["1", "2", "3", "4", "5"]:
        selected_route = list(self.routes.keys())[int(selected_route) - 1]

        # Validasi Jumlah Tiket
        tickets = self.get_valid_input("Masukkan jumlah tiket yang ingin dipesan: ",
                                       self.validate_ticket_count,
                                       "Masukkan angka yang valid untuk jumlah tiket.")
        tickets = int(tickets)

        total_price = tickets * self.routes[selected_route]

        # Validasi Nama Pemesan
        name = self.get_valid_input("Masukkan nama pemesan: ",
                                    self.validate_name,
                                    "Nama hanya boleh mengandung huruf dan spasi.")

        # Validasi Tanggal
        date = self.get_valid_input("Masukkan tanggal keberangkatan (YYYY-MM-DD): ",
                                    self.validate_date,
                                    "Format tanggal tidak valid. Gunakan format YYYY-MM-DD.")

        # Validasi Waktu
        time = self.get_valid_input("Masukkan waktu keberangkatan (HH:MM): ",
                                    self.validate_time,
                                    "Format waktu tidak valid. Gunakan format HH:MM.")

    self.bookings.append({
        "name": name,
        "route": selected_route,
        "tickets": tickets,
        "tanggal": date,
        ..
    })

```

```

        "waktu": time,
        "total": total_price
    })
    print(f"Tiket untuk {selected_route} berhasil dipesan! Total Harga: Rp {total_price}")
else:
    print("Pilihan rute tidak valid.")

def view_booking(self):
    if not self.bookings:
        print("Tidak ada tiket yang dipesan.")
    else:
        print("\nDaftar Tiket:")
        for index, booking in enumerate(self.bookings, start=1):
            print(f"{index}. {booking['name']} - {booking['route']} - {booking['tanggal']} - {booking['waktu']} - {booking['tickets']} tiket - Rp {booking['total']}")

def reschedule_booking(self):
    self.view_booking()
    if not self.bookings:
        return

    while True:
        try:
            booking_index = int(input("\nMasukkan nomor pemesanan yang ingin direschedule:
")) - 1
            if 0 <= booking_index < len(self.bookings):
                # Validasi Tanggal Baru
                new_date = self.get_valid_input("Masukkan tanggal keberangkatan baru (YYYY-MM-DD): ",
                                                self.validate_date,
                                                "Format tanggal tidak valid. Gunakan format YYYY-MM-DD.")

                # Validasi Waktu Baru
                new_time = self.get_valid_input("Masukkan waktu keberangkatan baru (HH:MM): ",
                                                self.validate_time,
                                                "Format waktu tidak valid. Gunakan format HH:MM.")

                self.bookings[booking_index]["tanggal"] = new_date
                self.bookings[booking_index]["waktu"] = new_time

```

```

print("Pemesanan berhasil direschedule!")
    break
else:
    print("Nomor pemesanan tidak valid.")
except ValueError:
    print("Input tidak valid. Masukkan nomor pemesanan yang benar.")

def main():
    system = BusTicketBooking()

    #Pilihan login atau daftar
    while True:
        choice = input("Apakah Anda sudah memiliki akun? (y/n): ")
        if choice.lower() == "y":
            if system.login():
                break
        elif choice.lower() == "n":
            system.register()
            break
        else:
            print("Pilihan tidak valid, silakan coba lagi.")

    while True:
        print("\nMenu:")
        print("1. Lihat Daftar Rute Bus")
        print("2. Pesan Tiket")
        print("3. Lihat Pemesanan")
        print("4. Reschedule Pemesanan")
        print("5. Keluar")

        choice = input("Pilih Menu: ")
        if choice == "1":
            system.display_routes()
        elif choice == "2":
            system.book_ticket()
        elif choice == "3":
            system.view_booking()
        elif choice == "4":
            system.reschedule_booking()
        elif choice == "5":

```



```
        print("Terima kasih telah menggunakan sistem kami!")
        break
    else:
        print("Pilihan tidak valid. Silahkan coba lagi.")

if __name__ == "__main__":
    main()
```

OUTPUT PROGRAM

1. In the first menu we will be given the choice whether we already have an account or not

Output

```
Apakah Anda sudah memiliki akun? (y/n):
```

2. if we choose not then it will be told to create an account and the minimum user name letters are five if it is less than the minimum, it will appear as shown on the screen.

Output

```
Apakah Anda sudah memiliki akun? (y/n): n
Masukkan username baru: Anto
Username tidak valid. Silahkan coba lagi.
```

3. but if we correctly enter the username, it will be asked to create a password and if successful, we are given five options and the bus route booking menu will appear.

Output

```
Apakah Anda sudah memiliki akun? (y/n): n
Masukkan username baru: Antonioo
Masukkan password baru: Antonioo1_

Registrasi berhasil!

Menu:
1. Lihat Daftar Rute Bus
2. Pesan Tiket
3. Lihat Pemesanan
4. Reschedule Pemesanan
5. Keluar
Pilih Menu: |
```

4. If we select number 1 on the booking menu, the listed routes will appear.

```
Menu:
1. Lihat Daftar Rute Bus
2. Pesan Tiket
3. Lihat Pemesanan
4. Reschedule Pemesanan
5. Keluar
Pilih Menu: 1

Daftar Rute dan Harga Tiket:
1. Jakarta-Bandung: Rp 150000
2. Jakarta-Surabaya: Rp 280000
3. Jakarta-Bali: Rp 500000
4. Jakarta-Yogyakarta: Rp 200000
5. Jakarta-Semarang: Rp 250000
```

5. If you want to book a ticket, select number two and a list of bus routes and prices will be displayed.

```
Menu:
1. Lihat Daftar Rute Bus
2. Pesan Tiket
3. Lihat Pemesanan
4. Reschedule Pemesanan
5. Keluar
Pilih Menu: 2

Daftar Rute dan Harga Tiket:
1. Jakarta-Bandung: Rp 150000
2. Jakarta-Surabaya: Rp 280000
3. Jakarta-Bali: Rp 500000
4. Jakarta-Yogyakarta: Rp 200000
5. Jakarta-Semarang: Rp 250000
```

6. After selecting number two and choosing the desired route we will be asked to fill in the number of tickets, the name of the booker, the date of departure, the time of departure and if it is appropriate then the ticket is successfully booked.

```
Pilih rute (1/2/3/4/5): 1
Masukkan jumlah tiket yang ingin dipesan: 1
Masukkan nama pemesan: Antonioo
Masukkan tanggal keberangkatan (YYYY-MM-DD): 2025-06-06
Masukkan waktu keberangkatan (HH:MM): 12:30
Tiket untuk Jakarta-Bandung berhasil dipesan! Total Harga: Rp 150000
```

7. if we have successfully booked a ticket and we choose number 3 on the menu then we can see the booking that we have ordered earlier.

```
Menu:
1. Lihat Daftar Rute Bus
2. Pesan Tiket
3. Lihat Pemesanan
4. Reschedule Pemesanan
5. Keluar
Pilih Menu: 3

Daftar Tiket:
1. Antonioo - Jakarta-Bandung - 2025-06-06 - 12:30 - 1 tiket - Rp
  150000
```

8. but if we want to change the departure schedule then choose number 4 and will be asked to fill out a form like the example below.

```
Menu:
1. Lihat Daftar Rute Bus
2. Pesan Tiket
3. Lihat Pemesanan
4. Reschedule Pemesanan
5. Keluar
Pilih Menu: 4

Daftar Tiket:
1. Antonioo - Jakarta-Bandung - 2025-06-06 - 12:30 - 1 tiket - Rp
  150000

Masukkan nomor pemesanan yang ingin direschedule: 1
Masukkan tanggal keberangkatan baru (YYYY-MM-DD): 2030-07-07
Masukkan waktu keberangkatan baru (HH:MM): 13:00
Pemesanan berhasil direschedule!
```

9. if we have finished booking bus tickets then select number 5 and will exit the menu.

```
Menu:
1. Lihat Daftar Rute Bus
2. Pesan Tiket
3. Lihat Pemesanan
4. Reschedule Pemesanan
5. Keluar
Pilih Menu: 5
Terima kasih telah menggunakan sistem kami!
```

CONCLUSION

The Bus E-Ticket project has been successfully developed as an online-based bus ticket booking system that offers convenience for users in making ticket reservations. With features such as schedule search, seat selection, and flexible payment methods, this system can increase efficiency and convenience for prospective passengers.

In addition, this system also provides benefits for transportation companies by allowing the management of booking data in a more structured manner. The data stored in the system can be used to analyze passenger travel patterns and improve service quality.

However, there are still some aspects that can be further developed, such as increasing system security with user data encryption features, developing a more interactive interface, and integration with wider electronic payment services. With further improvement and development, this system has the potential to become a more effective and efficient digital solution in the transportation industry.